

MTRX3760 - Project 1

Hello, Turtlebots!

This project contributes 10% towards your final mark. Total Marks: 100.

This assignment is due before the start of your allocated lab session in two weeks, i.e. before the start of the **Week 8 lab session for Weds/Fri labs**, and before the **start of the Week 9 lab session for Monday labs**. Late assignments will be subjected to the University's late submission policy unless accompanied by a valid special consideration form. Plagiarism will be dealt with in accordance with the University of Sydney plagiarism policy. **Lab submissions will not be accepted more than a week after the due date.**

Submissions will be assessed based on the following components. Incomplete submissions will have severely reduced marks.

- **Report:** Each group will submit a single .pdf report using the class Canvas site. The report should be very simple and brief, directly addressing the questions laid out in the assignment.

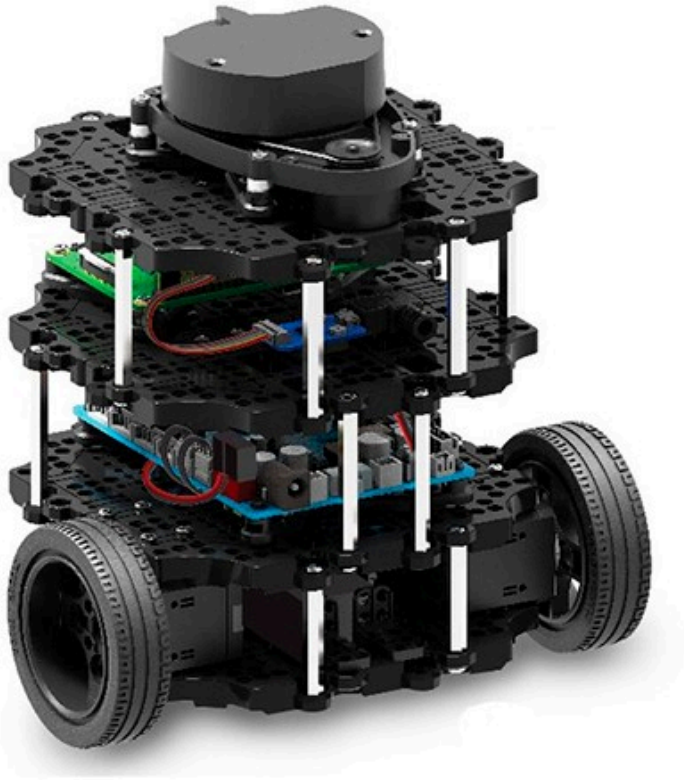
The front page of your report should include the SIDs and tutorial sections of all members of your group. Do not include names on the cover page to comply with the University's anonymous marking policy.

Code appendix: The appendix of your report **must** contain a **printout** (in text format) of all source code you wrote for the assignment. File header comments and proper formatting will be critical to making this section readable.

- **Code:** Submit your code (only code you wrote, and no binaries) in a .zip file via the canvas site.

Group Work

- You must complete this project in **groups of three or four** students [clarified 2024.Sept.04]
- Each group must nominate a leader who will submit the project to canvas for the group; only one group member should submit the work
- All group members must be in the same timetabled lab section
- Projects by larger groups will be graded with higher expectations



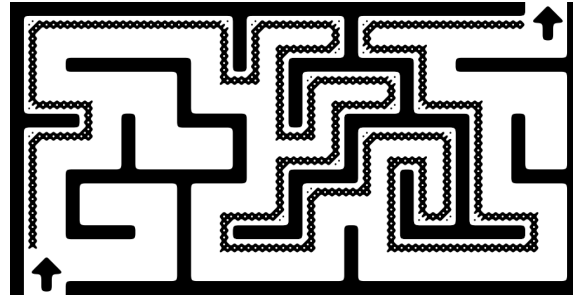
Maze Solver in Simulation

Simulate a Turtlebot 3 in RViz and Gazebo, including simulated laser and camera, by following the instructions here, up to and including Section 6.2:

<https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/>

Be careful to select the appropriate ROS version at the top of the page.

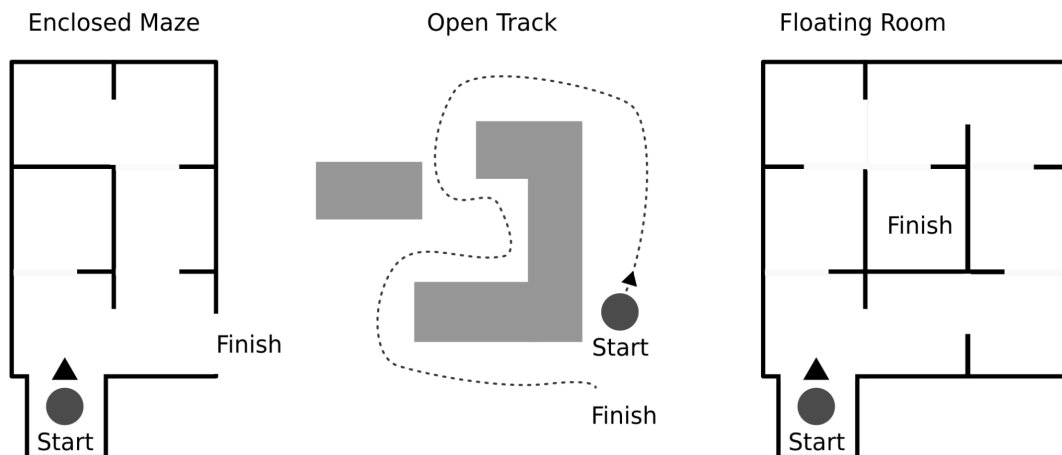
Now implement your own code that gives the turtlebot the ability to autonomously navigate through a complex space. Wall following is a classic approach to solving this kind of problem. The method works like it sounds: a robot chooses to either follow the wall to its right, or to its left, until it reaches the maze exit. Shown here is a left- wall-following robot solving a maze:



[adapted from <https://commons.wikimedia.org/wiki/File:Maze01-02.png>]

To add wall following capability to your project, locate and modify `turtlebot3_drive.cpp` so that instead of driving around aimlessly, the robot performs left wall following. Hint: start by looking inside `turtlebot3_drive.cpp`.

For this project you must demonstrate the functionality of your approach on at least two different types of maze. Some example types of maze are shown below. The maze above is an enclosed maze with no floating rooms.



Note that a “floating room” is a scenario in which the solution to the maze is not possible by only performing wall following.

You may make some simplifying assumptions, e.g. that the robot always begins adjacent to a wall to its left or to its right.

Code Refactor

Chances are your implementation process for the autonomous wall follower resulted in some less-than-ideal coding decisions. Using git to prevent doing irreversible damage, refactor (or rewrite) your code so that it is a well-structured object-oriented program. Consider how to best split up your code into classes, how best to split your classes into files, and which techniques covered in lecture are appropriate for making this program as clear and maintainable as possible.

Be careful that the example code for using ROS is often function-based and does not exhibit strong object-oriented design. You may wish to refer to the design checklists provided with the labs and their associated solution guides, and the coding principles and best practices taught in lectures, to guide you in deciding how to structure your program.

Submit only your final, refactored, working code.

Grading

[20 Marks] **Functionality and Testing**

In your report include images depicting your turtlebot simulation, showing that:

- The simulation works and includes **LiDAR** and **camera**
- Your solution can solve at least **two types of maze**, by showing the **trajectory** followed by the robot in at least two different scenarios

Screenshots are ideal here. You should not draw the path followed by the robot by hand, make the computer do the work. You should not photograph your screen with a camera, use screenshot software.

Be sure to include captions briefly describing what each figure in your report is showing.

The report and code in your attached .zip file will be assessed for correct functionality.

[40 Marks] **Code Quality and Development Process**

Revision Control: Include in your report your git commit history as reported by:

```
git log --graph --oneline
```

Note: copy/paste the text, screenshots are unacceptable

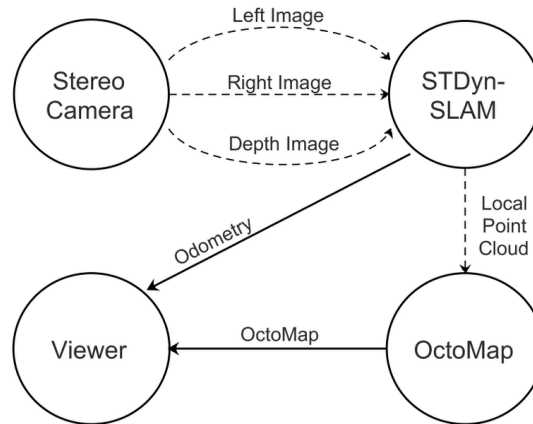
Teamwork: List in brief bullet-point form the contributions of each team member to the project.

The code, revision history, and report will be assessed for code quality and development process.

[40 Marks] **Design**

ROS Node Design Diagram: Draw a design diagram depicting how your program is split across ROS nodes. Each node should be in a circle, with arrows between nodes labelled with the key message(s) passed between them, like a typical top-level system diagram. Note that this is a *design* document and so `rtq_graph` or other automatically generated *diagnostic diagrams* will not be accepted.

The following shows an example, ignore the distinction between solid and dashed lines. Your diagram should have **no dashed lines**.

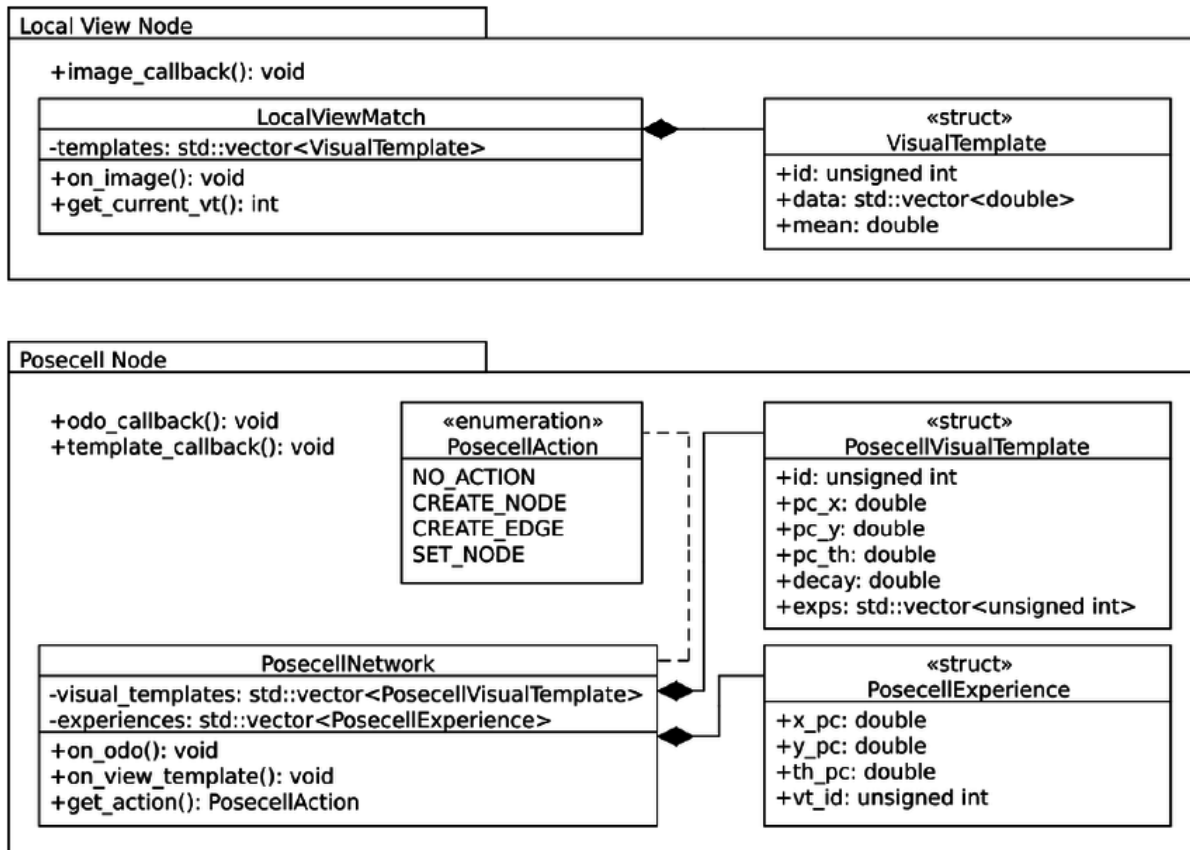


From Esparza et al "The STDyn-SLAM: A stereo vision and semantic segmentation approach for SLAM in dynamic outdoor environments", 2020.

UML Class Diagram: In your report provide a UML class diagram for the parts of your design that you programmed. You need not include member variables and functions.

Hints:

- **Do** include relationships between the classes you programmed and **classes in ROS**.
- In this unit we have emphasised composition (has-a), inheritance (is-a), and association (knows-a) relationships. These are *static* relationships: a car *always* has wheels, a student is *always* a person. Focus on getting these relationships right in your design and depicting them in your UML diagram.
- UML class diagram relationships don't exist between separate executables. Below is an example of a UML diagram for a program split across multiple nodes, each in its own executable.



Adapted from Ball et al. "OpenRatSLAM: an open source brain-based SLAM system", 2013

The code and report will be assessed for design quality including following the design principles taught in lecture.

Self-Assessment

[+5 Bonus] In your report estimate the level of achievement of your submission. Show estimates for all component grades as well as the total. If your estimated total is within 5 marks of the correct score you will be awarded 5 bonus marks. Ignore late penalties when estimating.